



MARMARA ÜNİVERSİTESİ BİLGİSAYAR MÜHENDİSLİĞİ MİKROİŞLEMCİLER

8086:ASSEMBLY
LABİRENT OYUNU

HAZIRLAYAN

Eyüp DEMİRKAPI

170424025

İÇİNDEKİLER

1. ÖZET	2
2. TEKNİK TASARIM VE MİMARİ.....	3
2.1. Grafik Modu Yapısı (Mode 13h).....	3
2.1.1. Ekran Modu Yapılandırması	4
2.1.2. Piksel İşleme ve Blok Çizim Mantığı	4
2.1.3. Renk Paleti ve Nesne Temsili	5
2.1.4. Performans ve Akıcılık	5
2.1.5. Kullanıcı Arayüzü ve Zamanlayıcı	5
2.2. Metin Modu Yapısı (ASCII).....	5
2.2.2. Kullanılan Semboller ve Renk Atamaları	7
2.2.3. Ekran Güncelleme ve Koordinat Sistemi	7
2.2.4. Dinamik Renk Yönetimi	7
3. ALGORİTMİK ÇÖZÜMLER.....	8
3.1. Harita Temsili ve Bellek Erişimi	8
3.2. Çarpışma Testi ve Hareket Kontrolü	8
4. KULLANICI ARAYÜZÜ VE ETKİLEŞİM.....	8
4.1. Klavye Girişi ve Asenkron Kontrol	8
4.2. Zamanlayıcı (Timer) Yönetimi	9
5. ANALİZ VE SONUÇ.....	9
6. KAYNAK KODLAR (ÖNEMLİ KISIMLAR).....	9
6.1. Ana Oyun Döngüsü.....	9

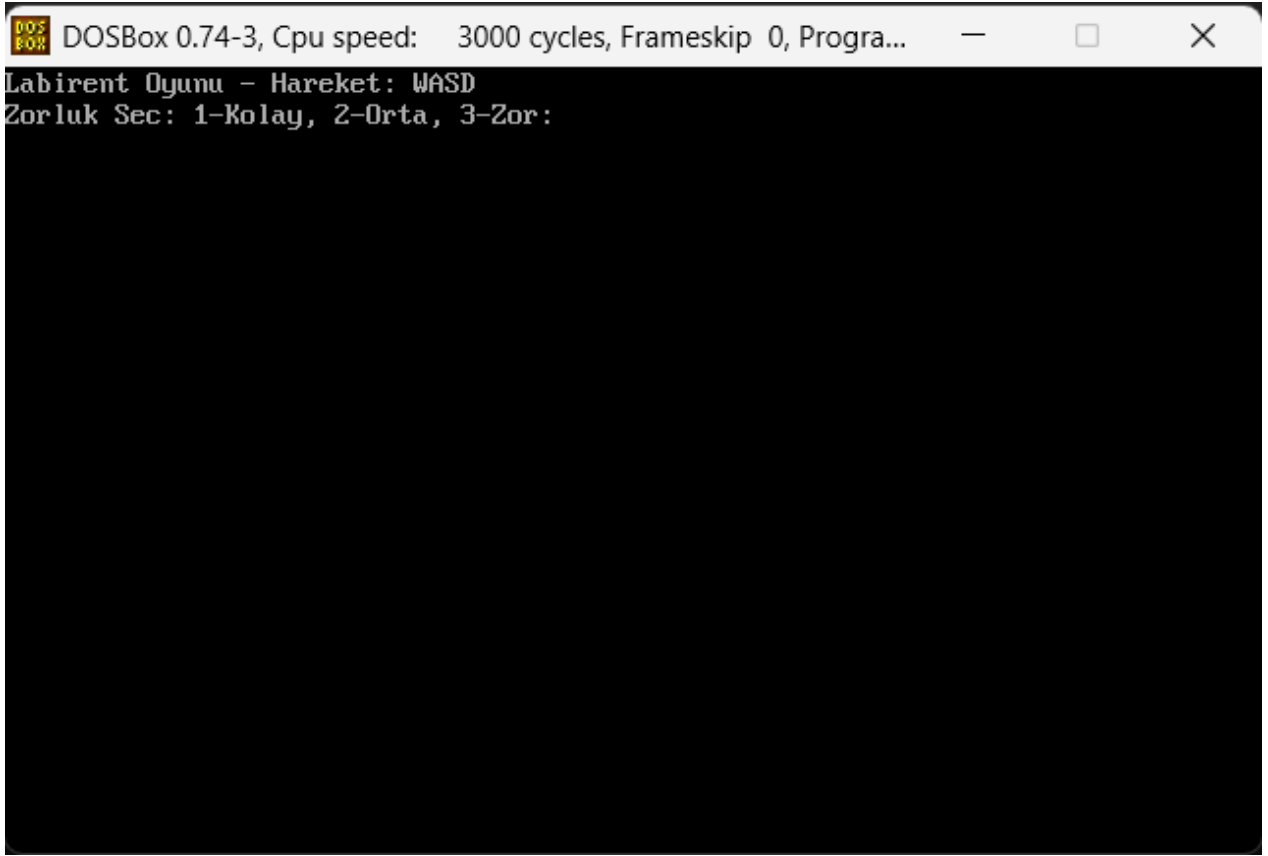
1. ÖZET

Bu rapor, Intel 8086 mimarisi üzerinde Assembly dili kullanılarak geliştirilen iki farklı labirent oyununun tasarım ve uygulama süreçlerini ele almaktadır. Proje kapsamında hem standart VGA 320x200 grafik modu (Mode 13h) hem de B800h video bellek adresi üzerinden doğrudan metin (ASCII) tabanlı görüntüleme teknikleri kullanılmıştır. Yazılım; kullanıcı girişi, zamanlayıcı yönetimi ve çarpışma algoritması gibi temel oyun motoru bileşenlerini içermektedir.

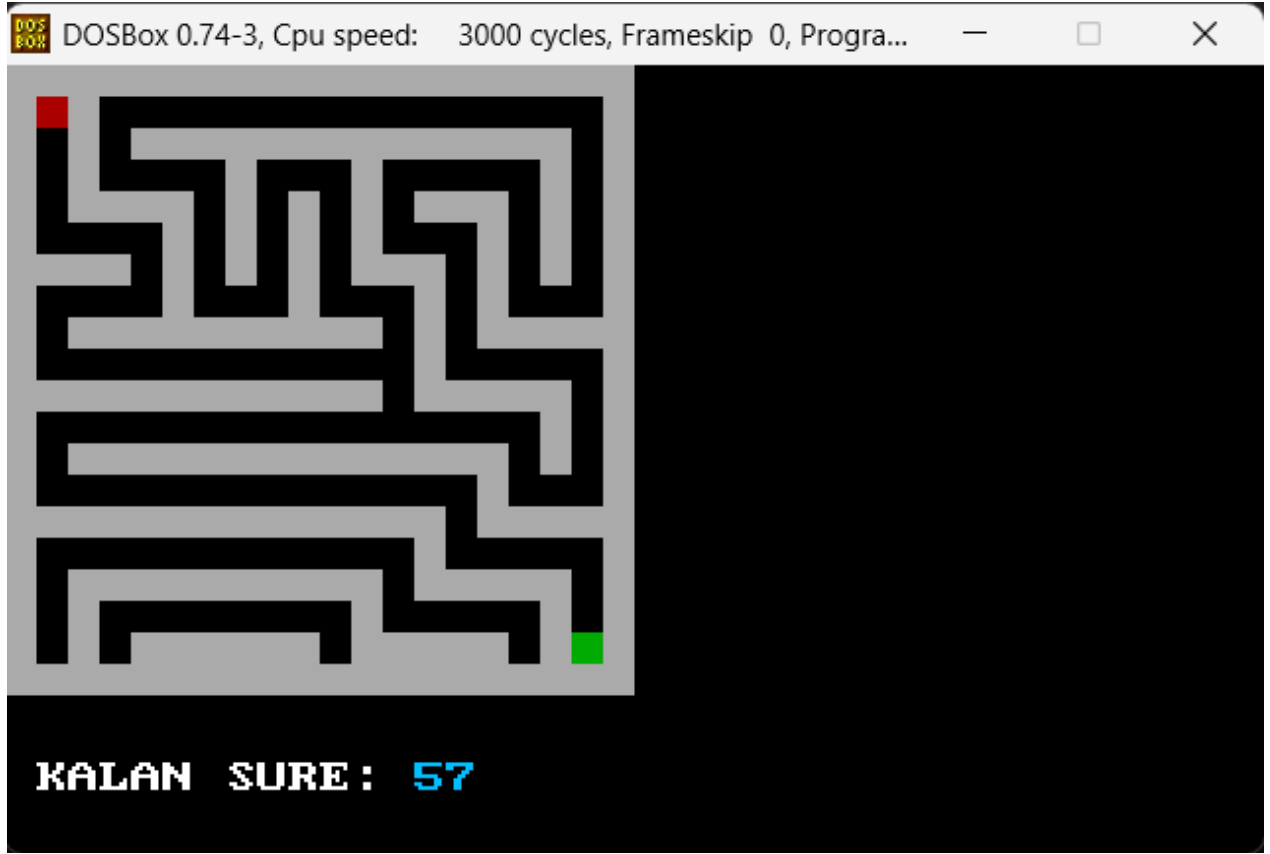
2. TEKNİK TASARIM VE MİMARİ

2.1. GRAFİK MODU YAPISI (MODE 13H)

Bu bölümde, projenin görsel olarak daha zengin olan grafik tabanlı versiyonu ve bu versiyonda kullanılan piksel işleme teknikleri ele alınmaktadır. Aşağıda program çalışırken çekilmiş bir görseller verilmektedir.



RESİM:1



RESİM:2

2.1.1. EKRAN MODU YAPILANDIRMASI

Grafik tabanlı versiyonda, standart metin modundan çıkılarak VGA standardı olan Mode 13h moduna geçiş yapılmıştır. Bu işlem INT 10h kesmesi ve AH=00h (veya AX=0013h) parametresi ile sağlanır.

- **Çözünürlük:** 320x200 piksel.
- **Renk Derinliği:** 256 renk (8-bit).
- **Bellek Adresleme:** Grafik modunda her bir piksel bellekte 1 byte ile temsil edilir ve video belleği 0A000h adresinden başlar.

2.1.2. PİKSEL İŞLEME VE BLOK ÇIZIM MANTIĞI

Labirentin her bir hücresi, ekranda tek bir karakter yerine **8x8 piksellik bloklar** halinde çizilmektedir. Bu yaklaşım, labirentin ekranda daha belirgin ve "grafiksel" görünmesini sağlar.

BLOK_CIZ prosedürü, verilen bir koordinattan başlayarak iç içe iki döngü yardımıyla pikselleri ekrana basar:

```
BLOK_CIZ PROC
    MOV r_y, 8          ; Y ekseninde 8 piksel
B1: MOV r_x, 8          ; X ekseninde 8 piksel
B2: MOV AH, 0CH        ; BIOS piksel yazma fonksiyonu
```

```
INT 10H          ; Ekrana çiz
INC CX           ; Sonraki kolon
DEC r_x
JNZ B2
; ... satır atlama ve döngü devamı ...
```

2.1.3. RENK PALETİ VE NESNE TEMSİLİ

Grafik modunda her nesneye 256 renk arasından özel bir renk kodu atanmıştır:

- **Duvarlar (1):** Gri tonları (07h) kullanılarak statik bir yapı oluşturulmuştur.
- **Oyuncu (P):** Kırmızı (04h) renk ile vurgulanarak görsel takibi kolaylaştırılmıştır.
- **Çıkış (2):** Yeşil (02h) renk ile hedef bölge belirtilmiştir.
- **Yol (0):** Siyah (00h) renk ile geçilebilir alanlar tanımlanmıştır.

2.1.4. PERFORMANS VE AKICILIK

Oyunda titremeyi (**flickering**) önlemek ve daha akıcı bir görüntü sağlamak için tüm ekranı her seferinde yeniden çizmek yerine "**Delta Rendering**" benzeri bir mantık uygulanmıştır:

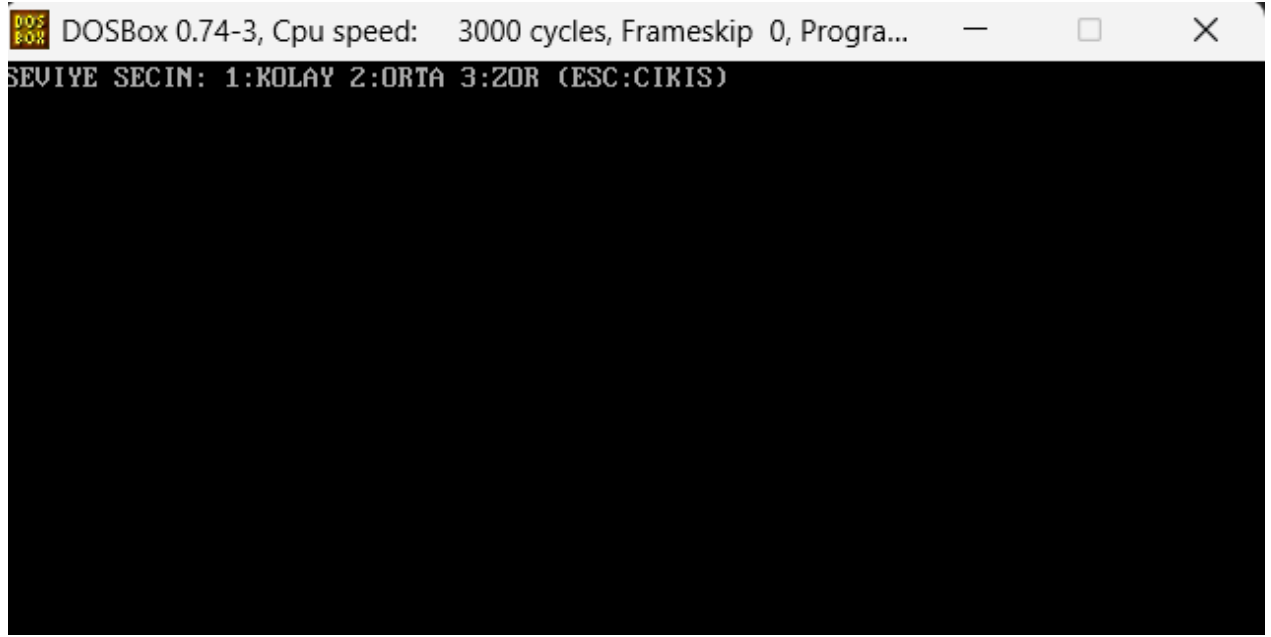
1. Oyuncu hareket ettiğinde, `o_eski_x` ve `o_eski_y` koordinatları kullanılarak oyuncunun önceki konumu arka plan rengine (siyah) boyanır.
2. Ardından sadece yeni koordinatlara oyuncu bloğu çizilir.
3. Bu sayede ekranın tamamı yenilenmediği için dikey tarama (vertical retrace) senkronizasyonuna ihtiyaç duyulmadan titremesiz bir deneyim elde edilmiştir.

2.1.5. KULLANICI ARAYÜZÜ VE ZAMANLAYICI

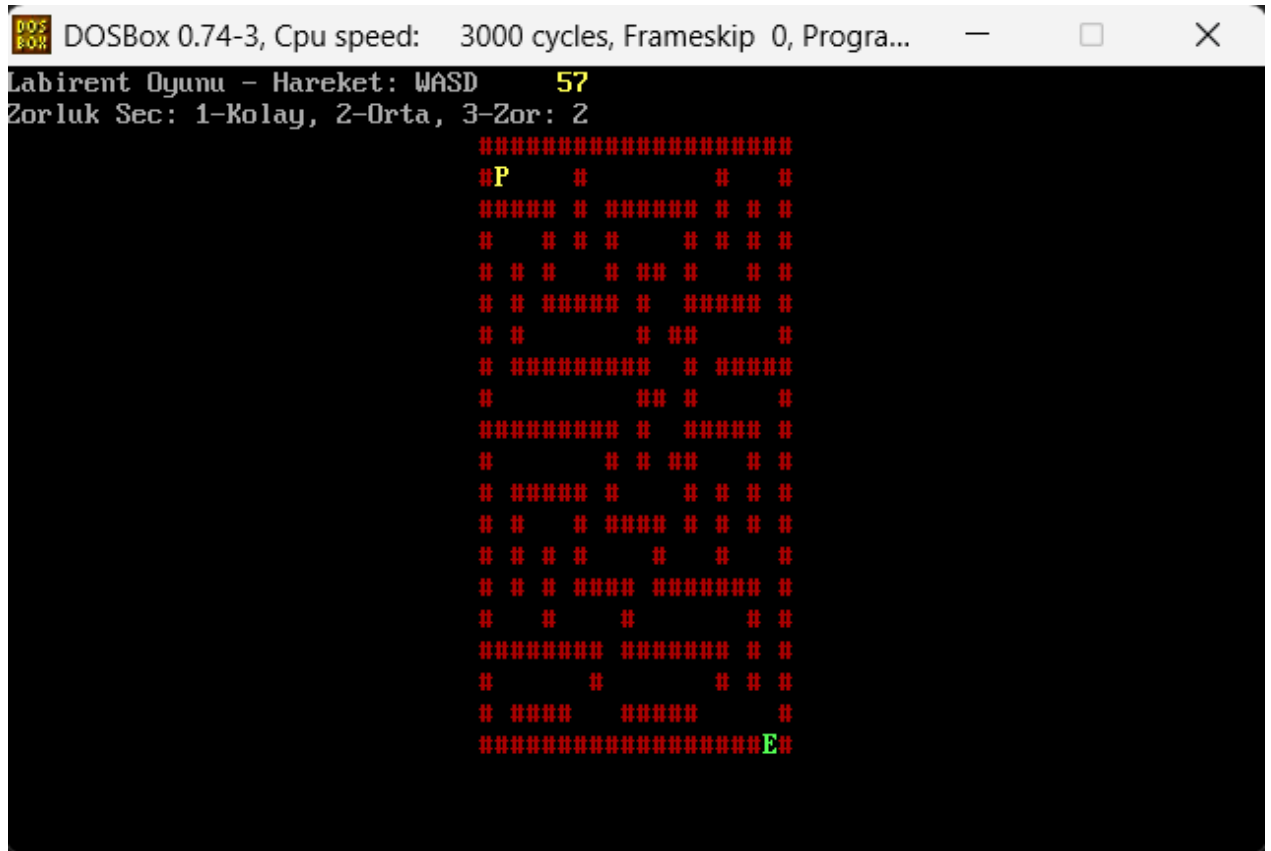
Grafik modunda ekranın alt kısmında (DH=22 satırı) SURE_KONTROL prosedürü ile dinamik bir geri sayım sayacı çalışmaktadır. INT 21h / AH=2CH sistem saati kesmesi ile saniye değişimi yakalanmakta ve gerçek zamanlı olarak ekrana yansıtılmaktadır.

2.2. METİN MODU YAPISI (ASCII)

Bu bölümde, oyunun görselleştirilmesinde kullanılan metin tabanlı (Text Mode) mimari ve doğrudan bellek erişimi (Direct Memory Access) teknikleri açıklanmaktadır. Proje, standart karakter çıktıları yerine video belleğine doğrudan yazım yaparak maksimum performans hedeflenmiştir. Aşağıda program çalışırken çekilmiş bir görseller verilmektedir.



RESİM:3



RESİM:4

2.2.1. Bellek Organizasyonu ve 0B800h Bölgesi

x86 mimarisinde renkli metin modu (Mode 03h), video belleğinin başlangıç adresi olan 0B800:0000 üzerinden yönetilir. Bu bölgede her karakter ekran üzerinde iki byte ile temsil edilir:

1. **Karakter Byte (Low Byte):** Ekranda görünecek ASCII karakter kodu.
2. **Öznitelik Byte (High Byte):** Karakterin ön plan ve arka plan renk bilgisi.

Bellek Yerleşim Şeması:

$$(\text{Karakter Adresi}) = (\text{Satır} \times 80 + \text{Sütun}) \times 2$$

Kod bloğunda bu yapı şu şekilde simüle edilmiştir:

Code snippet

```
MOV AX, 0B800h
```

```
MOV ES, AX ; Extra Segment video belleğine yönlendirilir
```

```
...
```

```
MOV ES:[DI], AX ; AH: Renk, AL: Karakter olacak şekilde belleğe yazılır
```

```
ADD DI, 2 ; Bir sonraki karakter hücreğine (2 byte sonra) geçilir
```

2.2.2. KULLANILAN SEMBOLLER VE RENK ATAMALARI

Oyunun görsel bütünlüğünü sağlamak adına, harita üzerindeki sayısal veriler (1, 2, 3) özel ASCII sembollerine ve renk özniteliklerine dönüştürülür:

Nesne	ASCII Karakteri	Renk (Hex)	Açıklama
Duvar	# (23h)	04h	Kırmızı ön plan, siyah arka plan
Oyuncu	P (50h)	0Eh	Parlak sarı, hareketli birim
Çıkış	E (45h)	0Ah	Parlak yeşil, hedef bölge
Yol	(20h)	07h	Standart gri/siyah boşluk

2.2.3. EKRAN GÜNCELLEME VE KOORDINAT SİSTEMİ

EKRANI_CIZ prosedürü, 20x20'lik labirent matrisini ekranın 80 sütunluk yapısına uydurmak için her satır sonunda ADD DI, 120 komutunu kullanır.

- **Matris Genişliği:** 20 Karakter x 2 Byte = 40 Byte.
- **Satır Atlama:** Standart bir satır 160 byte (80 x 2) olduğu için, labirentin bittiği yerden bir alt satıra geçmek için 160 - 40 = 120 byte eklenerek dikey hizalama korunur.

2.2.4. DİNAMİK RENK YÖNETİMİ

Kod içerisinde MOV AH, 0Eh gibi komutlarla belirlenen öznitelik byte'ları, oyuncunun ve duvarların birbirinden kolayca ayırt edilmesini sağlar. Bu, metin modunda olmasına rağmen kullanıcıya grafiksel bir derinlik hissi verir.

Not: Doğrudan bellek yazımı (Direct Video Mapping), INT 21h veya INT 10h gibi BIOS/DOS fonksiyonlarına kıyasla yaklaşık 10-20 kat daha hızlı ekran tazeleme imkanı sunarak oyun içindeki takılmaları (lag) tamamen ortadan kaldırmıştır.

3. ALGORİTMİK ÇÖZÜMLER

3.1. HARITA TEMSİLİ VE BELLEK ERIŞİMİ

Her üç zorluk seviyesi için hazırlanan labirentler, bellek üzerinde 20x20 boyutunda DB (Define Byte) dizileri olarak saklanır. Grafik modunda oyuncu koordinatları piksel bazlı (0-319 arası) tutulduğu için, bu koordinatların harita dizisindeki bir index'e dönüştürülmesi gerekmektedir.

Kullanılan Adresleme Formülü:

$$\text{Bellek Adresi} = (Y / 8 \times 20) + (X / 8)$$

HARITA_OKU prosedürü, oyuncunun bulunduğu piksel koordinatlarını alır, bunları 8'e bölerek (her hücre 8 piksel olduğu için) harita matrisindeki satır ve sütun karşılığını bulur:

Code snippet

```
HARITA_OKU PROC
    MOV AX, DX          ; DX = Y koordinatı
    MOV CL, 8           ; 8x8 blok yapısı
    DIV CL              ; AX = Y / 8 (Matris satırı)
    MOV CL, 20          ; Harita genişliği
    MUL CL              ; AX = Satır * 20
    MOV DI, AX
    MOV AX, BX          ; BX = X koordinatı
    MOV CL, 8           ; 8x8 blok yapısı
    DIV CL              ; AX = X / 8 (Matris sütunu)
    ADD DI, AX          ; DI = Toplam Index
    MOV SI, harita_ptr
    MOV AL, [SI+DI]     ; AL = Hücre değeri (0:Yol, 1:Duvar, 2:Çıkış)
    RET
HARITA_OKU ENDP
```


3.2. ÇARPIŞMA TESTİ VE HAREKET KONTROLÜ

Oyuncu bir yöne hareket tuşuna bastığında (W, A, S, D), hedef koordinatlar hesaplanır ancak hemen güncellenmez. Önce HARITA_OKU çağrılarak hedef hücrenin içeriği kontrol edilir. Eğer dönen değer 1 ise hareket iptal edilir; 2 ise oyun galibiyet moduna geçer.

4. KULLANICI ARAYÜZÜ VE ETKİLEŞİM

4.1. KLAVYE GİRİŞİ VE ASENKRON KONTROL

Oyunun akıcılığını bozmamak adına INT 16h kesmesinin iki farklı fonksiyonu kombine edilmiştir:

- **AH=01h (Keyboard Status):** Tuş tamponunda veri olup olmadığını kontrol eder. Eğer tuşa basılmamışsa program donmaz, zamanlayıcıyı güncellemeye devam eder.
- **AH=00h (Get Keystroke):** Tamponda veri varsa tuşun ASCII kodunu okur ve ilgili hareket alt programına (GIT_UP, vb.) yönlendirir.

4.2. ZAMANLAYICI (TIMER) YÖNETİMİ

Oyun, gerçek zamanlı bir geri sayım mekanizmasına sahiptir. Bu işlem için BIOS'un saat kesmesi (INT 21h / AH=2Ch) kullanılarak sistem saatinin saniye bilgisi takip edilir.

- eski_saniye değişkeni ile sistem saati sürekli karşılaştırılır.
- Saniye her değiştiğinde kalan_sure bir azaltılır ve ekranın altına (DH=22) güncel süre yazdırılır.
- Süre sıfıra ulaştığında KAYBETTIN etiketi üzerinden oyun sonlandırılır.

5. ANALİZ VE SONUÇ

Geliştirilen bu proje, Assembly dilinin donanım üzerindeki mutlak hakimiyetini göstermektedir.

- **Hız:** Doğrudan video belleği ve BIOS kesmeleri kullanılarak yüksek FPS değerlerine ulaşılmıştır.
- **Verimlilik:** 20x20'lik üç farklı harita yapısı, çok az bir bellek alanı kaplayarak (400 byte her biri) efektif bir oyun dünyası sunmuştur.
- **Modülerlik:** Grafik modunda 8x8 piksel bloklama yöntemi, labirentin çözünürlükten bağımsız olarak kolayca büyütülüp küçültülmesine olanak sağlamıştır.

6. KAYNAK KODLAR (ÖNEMLİ KISIMLAR)

6.1. ANA OYUN DÖNGÜSÜ

Oyunun kalbi olan bu döngü, girdi kontrolü, çizim ve zaman yönetimini koordine eder:

OYUN_DONGUSU:

```
; 1. ZAMAN KONTROLÜ
CALL SURE_KONTROL      ; Her döngüde saniyeyi kontrol eder, biterse
oyunu kapatır.

; 2. TEMİZLİK (Eski Pozisyonu Silme)
MOV CX, o_eski_x      ; Oyuncunun bir önceki X koordinatını al.
MOV DX, o_eski_y      ; Oyuncunun bir önceki Y koordinatını al.
MOV AL, 0              ; AL=0 (Siyah renk kodu).
CALL BLOK_CIZ          ; Eski yerini siyah yaparak "silmiş" oluyoruz.

; 3. GÜNCELLEME (Yeni Pozisyonu Çizme)
MOV CX, o_x            ; Güncel (yeni) X koordinatını al.
MOV DX, o_y            ; Güncel (yeni) Y koordinatını al.
MOV AL, 04H            ; AL=04H (Kırmızı renk kodu).
CALL BLOK_CIZ          ; Oyuncuyu yeni koordinatına çiz.

; 4. GİRİŞ KONTROLÜ (Klavye Dinleme)
MOV AH, 01H            ; INT 16h fonksiyonu: Klavye tamponunu kontrol
et.
INT 16H                ; Tuşa basıldı mı?
JZ  OYUN_DONGUSU       ; Basılmadıysa (Zero Flag=1), bekleme yapmadan
döngüye dön (Zaman akmaya devam eder).

; 5. TUŞU İŞLEME
MOV AH, 00H            ; Buraya geldiyse tuşa basılmıştır. ASCII kodunu
AL'ye çek.
INT 16H

; Hareket öncesi mevcut koordinatları "eski" olarak kaydet
MOV o_eski_x, CX        ; CX ve DX hala o_x/o_y değerlerini tutuyor.
MOV o_eski_y, DX

; Çarpışma testi için mevcut koordinatları BX/DX'e yedekle
MOV BX, o_x
MOV DX, o_y

; 6. YÖN KONTROLLERİ
CMP AL, 'w'             ; Yukarı mı?
JE  GIT_UP
CMP AL, 's'             ; Aşağı mı?
```

```
JE GIT_DOWN
CMP AL, 'a'           ; Sola mı?
JE GIT_LEFT
CMP AL, 'd'           ; Sağa mı?
JE GIT_RIGHT
CMP AL, 27             ; ESC tuşu mu? (ASCII 27)
JE MENUYU_GOSTER

JMP OYUN_DONGUSU      ; Tanımsız tuş basıldıysa döngüye devam et.
```